

OOP Basics

Object-Oriented Programming in Python

■ LEARNING OUTCOME

By the end of this module you will design and implement Python classes with attributes, methods, properties, inheritance, and special (dunder) methods – writing reusable, well-structured object-oriented code.

■ Skills You Will Gain

- Define classes with `__init__` and instance attributes
- Distinguish instance, class, and static methods
- Use properties for controlled attribute access
- Implement inheritance and method overriding
- Use special methods: `__str__`, `__repr__`, `__len__`, `__eq__`
- Understand multiple inheritance and MRO

■ OOP is how you model real-world problems in code. Classes, inheritance, encapsulation -- these patterns make large codebases manageable. Every major Python library (Django, pandas, SQLAlchemy) is built with OOP.

SECTION 1

Classes and Objects

```
class BankAccount: """A simple bank account with deposit, withdraw, and balance.""" # Class variable -- shared by ALL instances bank_name = "PyBank" total_accounts = 0 def __init__(self, owner: str, initial_balance: float = 0.0): """Initialize account with owner and optional balance.""" # Instance variables -- unique to each instance self.owner = owner self._balance = initial_balance # _ prefix = "private by convention" self.__id = BankAccount.total_accounts # __ prefix = name mangling BankAccount.total_accounts += 1 # Instance method -- receives self def deposit(self, amount: float) -> None: if amount <= 0: raise ValueError(f"Deposit amount must be positive, got {amount}") self._balance += amount print(f"Deposited {amount:.2f}. Balance: {self._balance:.2f}") def withdraw(self, amount: float) -> float: if amount > self._balance: raise ValueError(f"Insufficient funds: {self._balance:.2f} available") self._balance -= amount return amount # Property -- controlled attribute access @property def balance(self) -> float: return self._balance @balance.setter def balance(self, value: float): if value < 0: raise ValueError("Balance cannot be negative") self._balance = value # Class method -- receives cls, not self @classmethod def from_dict(cls, data: dict): """Alternative constructor from dictionary.""" return cls(data["owner"], data.get("balance", 0)) # Static method -- no self or cls @staticmethod def validate_amount(amount: float) -> bool: return isinstance(amount, (int, float)) and amount > 0 # Special (dunder) methods def __str__(self): return f"Account({self.owner}, {self._balance:.2f})" def __repr__(self): return f"BankAccount(owner={self.owner!r}, balance={self._balance!r})" def __len__(self): return int(self._balance) def __eq__(self, other): return isinstance(other, BankAccount) and self._balance == other._balance def __lt__(self, other): return self._balance < other._balance # USAGE account = BankAccount("Alice", 1000) account.deposit(500) account.withdraw(200) print(account) # calls __str__ print(account.balance) # uses property getter account.balance = 2000 # uses property setter acc2 = BankAccount.from_dict({"owner": "Bob", "balance": 500}) # class method
```

SECTION 2

Inheritance

```
class SavingsAccount(BankAccount): """Savings account with interest rate.""" def __init__(self, owner: str, balance: float = 0.0, rate: float = 0.05): super().__init__(owner, balance) # call parent __init__ self.rate = rate def add_interest(self) -> None: interest = self._balance * self.rate self.deposit(interest) print(f"Interest added: {interest:.2f}") def withdraw(self, amount: float) -> float: """Override: savings accounts have minimum balance.""" if self._balance - amount < 100: raise ValueError("Savings must maintain minimum balance of 100") return super().withdraw(amount) # call parent method def __str__(self): return f"SavingsAccount({self.owner}, {self._balance:.2f}, {self.rate:.1%})" class CheckingAccount(BankAccount): """Checking account with overdraft protection.""" def __init__(self, owner, balance=0, overdraft_limit=500): super().__init__(owner, balance) self.overdraft_limit = overdraft_limit def withdraw(self, amount): if amount > self._balance + self.overdraft_limit: raise ValueError("Exceeds overdraft limit") self._balance -= amount return amount # Checking inheritance print(isinstance(SavingsAccount("Alice"), BankAccount)) # True print(issubclass(SavingsAccount, BankAccount)) # True print(SavingsAccount.__mro__) # Method Resolution Order # Abstract base classes -- force subclasses to implement methods from abc import ABC, abstractmethod class Shape(ABC): @abstractmethod def area(self) -> float: ... @abstractmethod def perimeter(self) -> float: ... def describe(self): print(f"{self.__class__.__name__}: area={self.area():.2f}") class Circle(Shape): def __init__(self, radius): self.radius = radius def area(self): return 3.14159 * self.radius**2 def perimeter(self): return 2 * 3.14159 * self.radius c = Circle(5) c.describe()
```

SECTION 3

Dataclasses and Special Methods

```

from dataclasses import dataclass, field from typing import ClassVar # dataclass: auto-generates
__init__, __repr__, __eq__ @dataclass class Product: name: str price: float quantity: int = 0 tags:
list = field(default_factory=list) # mutable default # Class variable (not an instance field)
TAX_RATE: ClassVar[float] = 0.18 @property def total_value(self): return self.price * self.quantity
def __post_init__(self): if self.price < 0: raise ValueError("Price cannot be negative") self.name
= self.name.strip().title() # normalize name @dataclass(frozen=True) # immutable -- can be used as
dict key class Point: x: float y: float def distance(self, other): return ((self.x-other.x)**2 +
(self.y-other.y)**2)**0.5 # DUNDER METHODS reference class Container: def __init__(self, items):
self.items = list(items) def __len__(self): return len(self.items) def __getitem__(self, idx):
return self.items[idx] # c[0], c[1:3] def __setitem__(self, i,v): self.items[i] = v # c[0] = "x"
def __contains__(self, x): return x in self.items # "x" in c def __iter__(self): return
iter(self.items) # for x in c def __next__(self): pass def __add__(self, other): return
Container(self.items + other.items) def __bool__(self): return bool(self.items) # if c: def
__str__(self): return str(self.items) def __repr__(self): return f"Container({self.items!r})"

```

■ PRACTICE Q & A

Q1. What is the difference between `__str__` and `__repr__`?

A: `__str__` is for human-readable output (used by `print()` and `str()`). `__repr__` is for unambiguous developer output (used by `repr()` and in `REPL`). `repr()` should ideally be valid Python to recreate the object: `Point(x=3, y=4)`.

Q2. What is the difference between `@classmethod` and `@staticmethod`?

A: `@classmethod` receives 'cls' as first arg -- can access/modify class state. Used for alternative constructors. `@staticmethod` receives no automatic first arg -- utility function that logically belongs to the class but needs no class/instance data.

Q3. What is the difference between `_` and `__` prefixes for attributes?

A: Single underscore `_`: convention for 'internal/private' -- not enforced by Python. Double underscore `__`: triggers name mangling -- `_ClassName__attr`. This makes accidental access from subclasses harder. Neither is truly private.

Q4. What is an abstract method?

A: A method decorated with `@abstractmethod` in an ABC class that **MUST** be implemented by any non-abstract subclass. Attempting to instantiate a class with unimplemented abstract methods raises `TypeError`.

Q5. What is a dataclass and when should you use it?

A: `@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` from type-annotated class attributes. Use for simple data-holding classes (like records/structs). Reduces boilerplate compared to writing all dunder methods manually.

■■ OOP Cheat Sheet	
<code>class MyClass:</code>	Define a class
<code>def __init__(self, x):</code>	Constructor
<code>self.attr = x</code>	Instance attribute
<code>ClassName.class_var</code>	Class variable
<code>@property</code>	Controlled attribute access
<code>@classmethod</code> / <code>@staticmethod</code>	Alternative constructors / utilities
<code>class Child(Parent):</code>	Inheritance
<code>super().__init__()</code>	Call parent constructor
<code>def __str__(self):</code>	Human-readable string
<code>def __repr__(self):</code>	Developer representation
<code>@dataclass</code>	Auto-generate boilerplate
<code>from abc import ABC, abstractmethod</code>	Abstract base class